

Supervised Learning Capstone: Ames Housing Price Prediction

Checkpoint 3 – Final Delivery and Benchmarking Report

Machine Learning Track

June 21, 2026

Abstract

This report presents the final deliverables for the Ames Housing supervised learning capstone project. Six algorithms – Linear Regression, Polynomial Regression, Logistic Regression, Support Vector Machines (Linear and RBF kernels), Decision Trees, and Random Forests – were implemented to predict house sale prices (regression) and a derived price category (classification: Budget / Standard / Premium). For each model, hyperparameter tuning curves were generated to diagnose overfitting and underfitting, and performance was benchmarked using RMSE / R^2 for regression tasks and Accuracy / F1-Score for classification tasks. The Random Forest and Polynomial Regression (degree 2) models achieved the strongest predictive performance for the regression task, while the SVM (RBF Kernel) and Random Forest models led the classification task, with the Polynomial Regression model also exhibiting pronounced overfitting that is discussed in detail.

Contents

1	Introduction	3
2	Theory	3
2.1	Linear Regression	3
2.2	Polynomial Regression	4
2.3	Logistic Regression	4
2.4	Support Vector Machines (SVM)	4
2.5	Decision Trees	4
2.6	Random Forests	5
3	Implementation Code	5
3.1	Linear Regression	5
3.2	Polynomial Regression	6
3.3	Logistic Regression	6
3.4	Support Vector Machine – Linear Kernel	7
3.5	Support Vector Machine – RBF Kernel	7
3.6	Decision Trees	7
3.7	Random Forest	8
4	Results and Findings	9
4.1	Linear Regression	9
4.2	Polynomial Regression (Degree 2)	10
4.3	Logistic Regression	10

4.4	Support Vector Machines	11
4.4.1	Linear Kernel	11
4.4.2	RBF Kernel	12
4.5	Decision Trees	13
4.6	Random Forest	15
4.7	Final Benchmarking Table	15
5	Conclusion	16

1 Introduction

The Ames Housing dataset is a widely used benchmark in supervised machine learning, containing 79 explanatory variables describing nearly every physical aspect of residential homes sold in Ames, Iowa. This capstone project uses the dataset to build a complete, end-to-end supervised learning pipeline covering both **regression** (predicting the continuous `SalePrice`) and **classification** (predicting a derived categorical target, `Price_Category`, obtained by binning `SalePrice` into *Budget*, *Standard*, and *Premium* tiers).

The objective of Checkpoint 3 is to consolidate the work completed across the earlier checkpoints (problem framing, data cleaning, feature engineering, scaling, and encoding) into a final, benchmarked comparison of six supervised learning algorithms:

1. Linear Regression
2. Polynomial Regression (degree 2)
3. Logistic Regression (multinomial)
4. Support Vector Machines – Linear and RBF kernels
5. Decision Trees (with cost-complexity pruning)
6. Random Forests (ensemble of decision trees)

For every model, hyperparameter tuning curves (Training vs. Validation performance) were plotted to visually diagnose the bias–variance trade-off, and the best-performing configuration was selected via 5-fold cross-validation or a held-out test split. The remainder of this report is organized as follows: Section 2 briefly reviews the theoretical foundations of each algorithm, Section 3 presents the implementation code, Section 4 reports results and findings (including the final benchmarking table), and Section 5 concludes with a summary of insights and recommendations.

Note on evaluation protocol: Linear Regression, Polynomial Regression, and Logistic Regression were evaluated on a single held-out 80/20 train-test split. SVM, Decision Tree, and Random Forest models were tuned and scored using 5-fold cross-validation (via `GridSearchCV`) over the entire dataset, since the primary goal there was hyperparameter selection rather than final hold-out testing. Where a metric was not directly computed in the corresponding notebook (e.g., test-set RMSE for SVM), this is noted explicitly in Section 4.

2 Theory

This section briefly summarizes the theoretical basis of each algorithm used in the pipeline.

2.1 Linear Regression

Linear Regression models the target variable as a linear combination of input features:

$$\hat{y} = \beta_0 + \sum_{i=1}^n \beta_i x_i$$

Coefficients β_i are estimated by minimizing the Ordinary Least Squares (OLS) cost function, i.e., the sum of squared residuals between predicted and actual values. It assumes linearity, independence of errors, homoscedasticity, and minimal multicollinearity among features.

2.2 Polynomial Regression

Polynomial Regression extends linear regression by introducing polynomial terms (and, when using `PolynomialFeatures`, interaction terms) of the original features, allowing the model to fit non-linear relationships while remaining linear in the transformed parameter space:

$$\hat{y} = \beta_0 + \sum_{i=1}^n \beta_i x_i + \sum_{i=1}^n \sum_{j \geq i}^n \beta_{ij} x_i x_j$$

While this increases model flexibility, expanding every feature to degree 2 dramatically increases dimensionality (quadratic in the number of original features), which significantly raises the risk of overfitting when the number of engineered features approaches or exceeds the number of training samples.

2.3 Logistic Regression

Logistic Regression is a linear model adapted for classification. For the multinomial (multi-class) case used here, the softmax function converts a linear score per class into a probability distribution:

$$P(y = k \mid x) = \frac{e^{\beta_k^T x}}{\sum_{j=1}^K e^{\beta_j^T x}}$$

Model parameters are estimated by maximizing the multinomial log-likelihood (equivalently, minimizing cross-entropy loss), typically using iterative optimization such as L-BFGS.

2.4 Support Vector Machines (SVM)

SVMs find the hyperplane that maximizes the margin between classes. For non-linearly separable data, the **kernel trick** implicitly projects data into a higher-dimensional space:

- **Linear Kernel:** $K(x_i, x_j) = x_i^T x_j$ – suitable for linearly separable or near-linear data.
- **RBF (Gaussian) Kernel:** $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$ – captures non-linear decision boundaries.

The regularization parameter C controls the trade-off between margin width and misclassification penalty (low C = wider margin, more tolerance for errors; high C = narrower margin, less tolerance). The kernel parameter γ controls the influence radius of a single training example in the RBF kernel.

2.5 Decision Trees

Decision Trees recursively partition the feature space into regions that minimize impurity at each split – using variance reduction (MSE) for regression or Gini impurity / entropy for classification. Unconstrained trees tend to overfit by memorizing training data. **Cost-Complexity Pruning** (controlled by α , `ccp_alpha`) penalizes tree complexity, removing weak branches to balance bias and variance:

$$R_\alpha(T) = R(T) + \alpha |T|$$

where $R(T)$ is the total impurity of the tree and $|T|$ is the number of terminal (leaf) nodes.

2.6 Random Forests

Random Forests are ensembles of decision trees trained on bootstrap-resampled subsets of the data (bagging), with each split considering only a random subset of features (`max_features`). Predictions are aggregated by averaging (regression) or majority voting (classification). This reduces variance relative to a single decision tree without substantially increasing bias, generally yielding more robust and generalizable models.

3 Implementation Code

This section presents the core implementation for each model, exactly as executed in the accompanying Jupyter notebooks. All models use the preprocessed, encoded, and scaled dataset `df_encoded_standard.csv` produced in Checkpoint 2.

3.1 Linear Regression

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.model_selection import train_test_split
4 from sklearn.linear_model import LinearRegression
5 from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
6
7 # 1. Load the preprocessed dataset
8 df = pd.read_csv('df_encoded_standard.csv')
9
10 # 2. Separate features (X) and target variable (y)
11 X = df.drop(columns=['SalePrice', 'Price_Category'])
12 y = df['SalePrice']
13
14 # 3. Train-Test Split (80% training, 20% testing)
15 X_train, X_test, y_train, y_test = train_test_split(
16     X, y, test_size=0.2, random_state=42)
17
18 # 4. Initialize and Train the Linear Regression Model
19 lr_model = LinearRegression()
20 lr_model.fit(X_train, y_train)
21
22 # 5. Make Predictions
23 y_pred_train = lr_model.predict(X_train)
24 y_pred_test = lr_model.predict(X_test)
25
26 # 6. Evaluate the Model
27 def evaluate_predictions(y_true, y_pred, dataset_name):
28     mse = mean_squared_error(y_true, y_pred)
29     rmse = np.sqrt(mse)
30     mae = mean_absolute_error(y_true, y_pred)
31     r2 = r2_score(y_true, y_pred)
32     print(f"--- {dataset_name} Performance ---")
33     print(f"R2 Score: {r2:.4f}")
34     print(f"MAE: {mae:.4f}")
35     print(f"RMSE: {rmse:.4f}")
36
37 evaluate_predictions(y_train, y_pred_train, "Training Set")
38 evaluate_predictions(y_test, y_pred_test, "Test Set")
```

3.2 Polynomial Regression

```

1 from sklearn.preprocessing import PolynomialFeatures
2 from sklearn.linear_model import LinearRegression
3 from sklearn.metrics import mean_squared_error, r2_score
4
5 X = df.drop(columns=['SalePrice', 'Price_Category'], errors='ignore')
6 y = df['SalePrice']
7
8 X_train, X_test, y_train, y_test = train_test_split(
9     X, y, test_size=0.2, random_state=42)
10
11 print(f"Original feature count: {X_train.shape[1]}")
12
13 # Apply Polynomial Features (Degree 2) to every column
14 poly = PolynomialFeatures(degree=2, include_bias=False)
15 X_train_poly = poly.fit_transform(X_train)
16 X_test_poly = poly.transform(X_test)
17
18 print(f"Expanded polynomial feature count: {X_train_poly.shape[1]}")
19
20 # Fit pure Linear Regression on the expanded feature space
21 lr_model = LinearRegression()
22 lr_model.fit(X_train_poly, y_train)
23
24 y_pred_train = lr_model.predict(X_train_poly)
25 y_pred_test = lr_model.predict(X_test_poly)

```

3.3 Logistic Regression

```

1 from sklearn.linear_model import LogisticRegression
2 from sklearn.metrics import (classification_report, accuracy_score,
3                             confusion_matrix, ConfusionMatrixDisplay)
4
5 X = df.drop(columns=['SalePrice', 'Price_Category'], errors='ignore')
6 y = df['Price_Category']
7
8 # Stratified split preserves class proportions
9 X_train, X_test, y_train, y_test = train_test_split(
10     X, y, test_size=0.2, random_state=42, stratify=y)
11
12 # Fit Multinomial Logistic Regression
13 log_reg = LogisticRegression(max_iter=1000, random_state=42)
14 log_reg.fit(X_train, y_train)
15
16 y_pred = log_reg.predict(X_test)
17
18 accuracy = accuracy_score(y_test, y_pred)
19 print(f"Overall Accuracy: {accuracy:.4f}")
20 print(classification_report(y_test, y_pred))
21
22 labels_order = ['Budget', 'Standard', 'Premium']
23 cm = confusion_matrix(y_test, y_pred, labels=labels_order)
24 disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels_order)
25 disp.plot(cmap='Blues', values_format='d')

```

3.4 Support Vector Machine – Linear Kernel

```

1 from sklearn.model_selection import GridSearchCV
2 from sklearn.svm import SVC
3
4 X = df.drop(columns=['SalePrice', 'Price_Category'], errors='ignore')
5 y = df['Price_Category']
6
7 c_values = [0.001, 0.01, 0.1, 1, 10, 100]
8 param_grid = {'C': c_values}
9
10 svm_model = SVC(kernel='linear', random_state=42)
11 grid_search = GridSearchCV(svm_model, param_grid, cv=5,
12                             return_train_score=True, n_jobs=-1)
13 grid_search.fit(X, y)
14
15 results = grid_search.cv_results_
16 train_accuracies = results['mean_train_score']
17 val_accuracies = results['mean_test_score']
18
19 print(f"Optimal C: {grid_search.best_params_['C']}")
20 print(f"Best Validation Accuracy: {grid_search.best_score_:.4f}")

```

3.5 Support Vector Machine – RBF Kernel

```

1 c_values = [0.1, 1, 10, 100]
2 gamma_values = ['scale', 'auto', 0.001, 0.01, 0.1, 1]
3 param_grid = {'C': c_values, 'gamma': gamma_values}
4
5 svm_rbf = SVC(kernel='rbf', random_state=42)
6 grid_search = GridSearchCV(svm_rbf, param_grid, cv=5,
7                             return_train_score=True, n_jobs=-1)
8 grid_search.fit(X, y)
9
10 best_params = grid_search.best_params_
11 best_score = grid_search.best_score_
12
13 print(f"Best Params: {best_params}, Accuracy: {best_score:.4f}")

```

3.6 Decision Trees

```

1 from sklearn.tree import DecisionTreeRegressor, DecisionTreeClassifier
2
3 # --- Regression: SalePrice ---
4 X = df.drop(columns=['SalePrice', 'Price_Category'], errors='ignore')
5 y = df['SalePrice']
6
7 refined_alphas = np.logspace(4, 9, num=25)
8 refined_alphas = np.insert(refined_alphas, 0, 0.0)
9
10 dt_regressor = DecisionTreeRegressor(random_state=42)
11 param_grid = {'cgp_alpha': refined_alphas}
12 grid_search = GridSearchCV(dt_regressor, param_grid, cv=5,
13                             scoring='r2', return_train_score=True, n_jobs=-1)
14 grid_search.fit(X, y)

```

```

15
16 print(f"Optimal Alpha: {grid_search.best_params_['ccp_alpha']:.2f}")
17 print(f"Optimal Validation R2: {grid_search.best_score_:.4f}")
18
19 # --- Classification: Price_Category ---
20 y_clf = df['Price_Category']
21 path_full = DecisionTreeClassifier(random_state=42).cost_complexity_pruning_path(X
22 , y_clf)
23 alphas_full = path_full.ccp_alphas
24
25 clf_dt = DecisionTreeClassifier(random_state=42)
26 param_grid = {'ccp_alpha': alphas_full}
27 grid_search = GridSearchCV(clf_dt, param_grid, cv=5,
28                             scoring='accuracy', return_train_score=True, n_jobs
29                             =-1)
30 grid_search.fit(X, y_clf)
31
32 print(f"Optimal Alpha: {grid_search.best_params_['ccp_alpha']:.5f}")
33 print(f"Optimal CV Accuracy: {grid_search.best_score_:.4f}")

```

3.7 Random Forest

```

1 from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier
2
3 # --- Regression: SalePrice ---
4 X = df.drop(columns=['SalePrice', 'Price_Category'], errors='ignore')
5 y = df['SalePrice']
6
7 param_grid = {
8     'n_estimators': [100, 200],
9     'max_depth': [10, 20, None],
10    'max_features': ['sqrt', 0.3]
11 }
12
13 rf_regressor = RandomForestRegressor(random_state=42)
14 grid_search = GridSearchCV(rf_regressor, param_grid, cv=5,
15                             scoring='r2', return_train_score=True, n_jobs=-1)
16 grid_search.fit(X, y)
17
18 best_rf = grid_search.best_estimator_
19 importances = best_rf.feature_importances_
20 print(f"Optimal Params: {grid_search.best_params_}")
21 print(f"Optimal CV R2: {grid_search.best_score_:.4f}")
22
23 # --- Classification: Price_Category ---
24 y_clf = df['Price_Category']
25 param_grid_clf = {
26     'n_estimators': [100, 200],
27     'max_depth': [10, 20, None],
28     'max_features': ['sqrt', 0.3],
29     'criterion': ['gini', 'entropy']
30 }
31
32 rf_classifier = RandomForestClassifier(random_state=42)
33 grid_search = GridSearchCV(rf_classifier, param_grid_clf, cv=5,
34                             scoring='accuracy', return_train_score=True, n_jobs
35                             =-1)

```

```

35 grid_search.fit(X, y_clf)
36
37 print(f"Optimal Params: {grid_search.best_params_}")
38 print(f"Optimal CV Accuracy: {grid_search.best_score_:.4f}")

```

4 Results and Findings

4.1 Linear Regression

The baseline Linear Regression model achieved an R^2 of 0.9204 and RMSE of \$21,783.51 on the training set, dropping to $R^2 = 0.7473$ and $RMSE = \$44,025.67$ on the test set. The substantial gap between training and test performance indicates moderate overfitting, likely driven by the large number of one-hot encoded categorical features relative to the number of training samples.

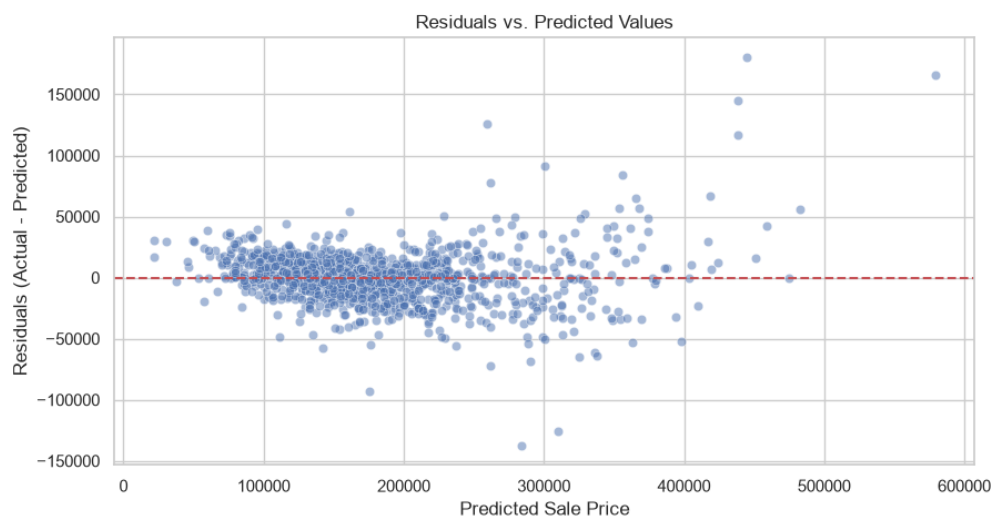
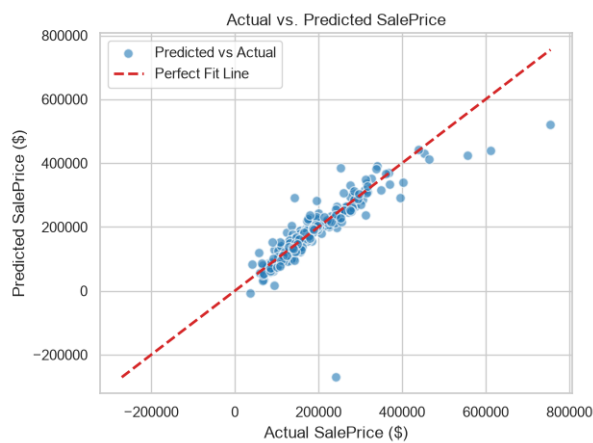
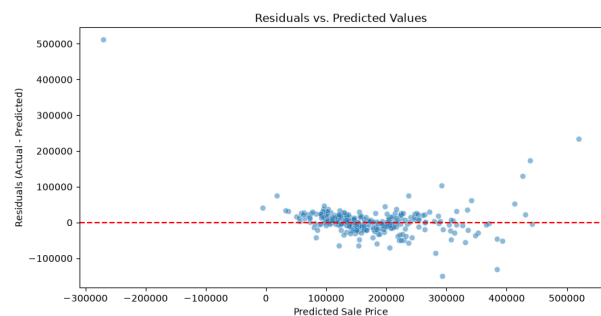


Figure 1: Linear Regression: Actual vs. Predicted SalePrice (Training Set)



(a) Training residuals



(b) Test residuals

Figure 2: Linear Regression residual plots. Mild funneling at higher predicted prices suggests minor heteroscedasticity.

The dominant positive coefficients corresponded to **RoofMat1** categories and **GarageCond** levels, while the most negative coefficients were associated with certain **GarageQual** and **Condition2** categories – though these large coefficient magnitudes on rare categorical dummies are themselves a symptom of high-dimensional overfitting rather than necessarily meaningful price drivers.

4.2 Polynomial Regression (Degree 2)

Expanding all 224 original features to degree 2 produced 25,424 polynomial features. This is the most overfit model in the benchmark: training RMSE collapsed to \$0.00 ($R^2 = 1.0000$), while test performance was $R^2 = 0.8690$ and $\text{RMSE} = \$31,695.96$.

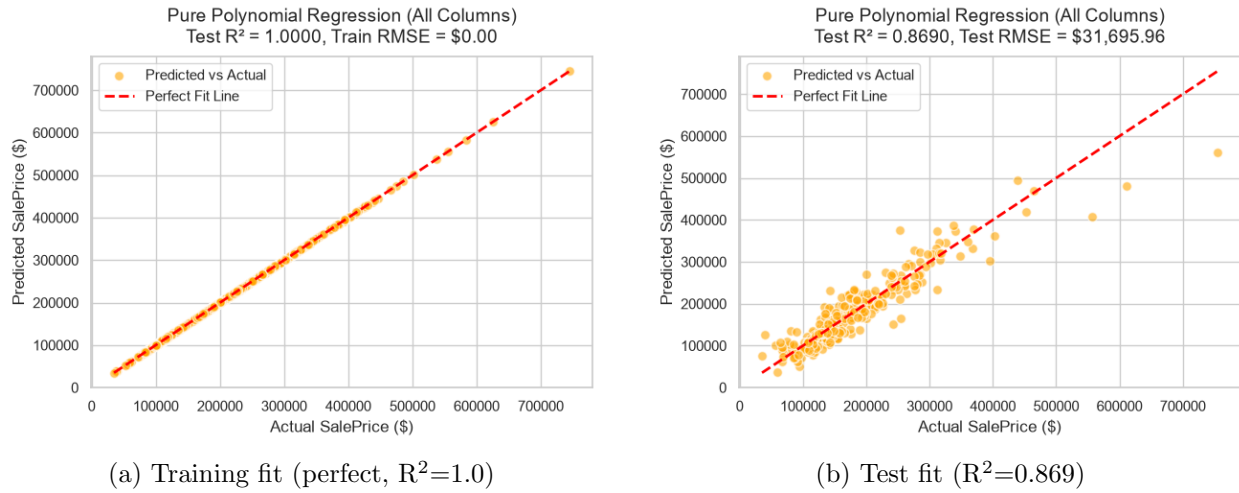


Figure 3: Polynomial Regression: a textbook overfitting signature – a perfect training fit fails to generalize.

This result is the clearest overfitting/underfitting demonstration in the entire benchmark: with 25,424 features and only $\sim 1,168$ training rows, the model has more parameters than data points, allowing it to perfectly interpolate the training set while still achieving a reasonable (though clearly inflated-confidence) test R^2 . In practice, this configuration would require strong regularization (Ridge/Lasso) or feature selection to be considered production-ready.

4.3 Logistic Regression

The multinomial Logistic Regression classifier achieved an overall test accuracy of 0.8596 and a weighted-average F1-score of 0.86. Per-class performance was strongest for the *Budget* category ($F1 = 0.91$) and weakest for the *Premium* category ($F1 = 0.74$), reflecting class imbalance (only 23 Premium samples in the test set vs. 165 Budget samples).

Table 1: Logistic Regression Classification Report

Class	Precision	Recall	F1-Score	Support
Budget	0.91	0.91	0.91	165
Standard	0.81	0.81	0.81	104
Premium	0.74	0.74	0.74	23
Accuracy	0.8596			
Weighted Avg	0.86	0.86	0.86	292

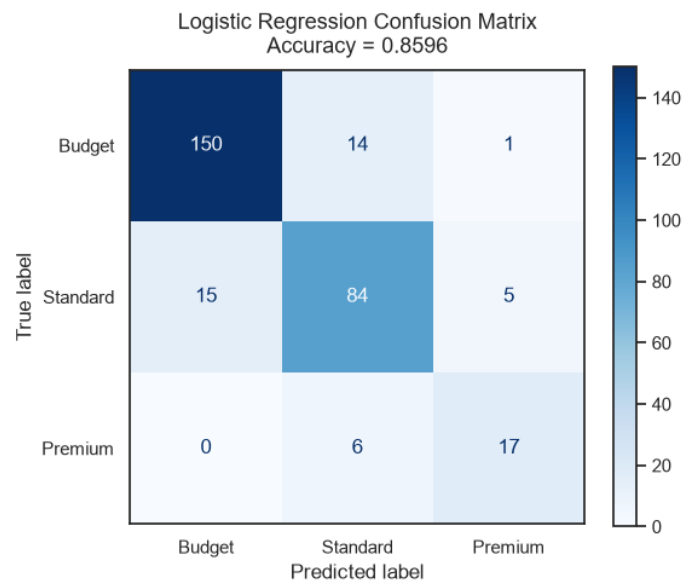


Figure 4: Logistic Regression confusion matrix on the test set.

Most misclassifications occur between adjacent classes (Budget↔Standard, Standard↔Premium), which is expected since `Price_Category` is derived from binning a continuous variable – borderline houses near a bin edge are inherently harder to classify correctly.

4.4 Support Vector Machines

4.4.1 Linear Kernel

A 5-fold cross-validated grid search over $C \in \{0.001, 0.01, 0.1, 1, 10, 100\}$ identified $C = 0.1$ as optimal, with validation accuracy of 0.8973.

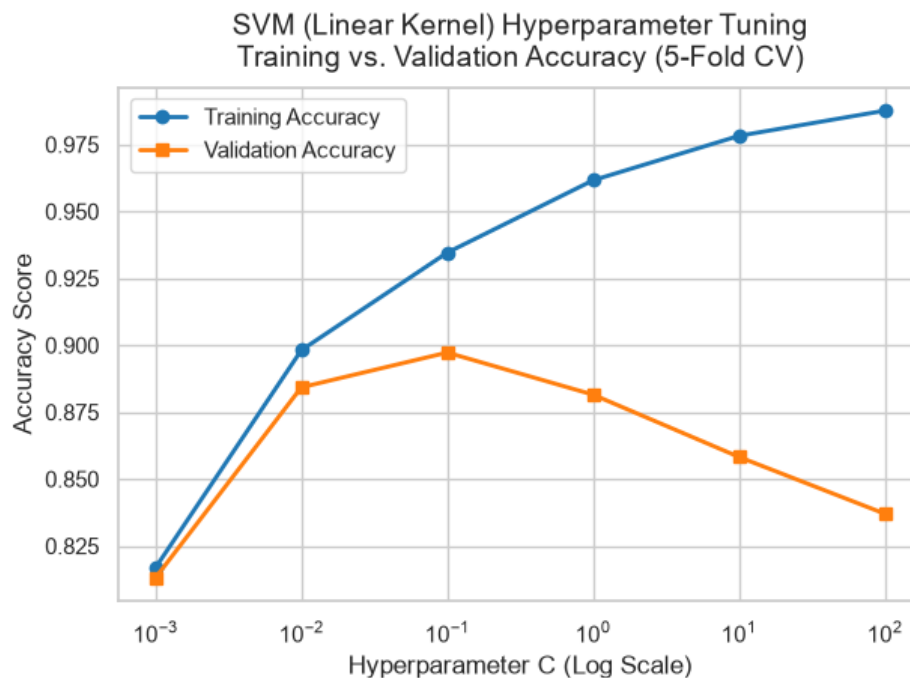


Figure 5: SVM (Linear Kernel): Training vs. Validation accuracy across C . Training accuracy rises monotonically with C while validation accuracy peaks near $C = 0.1$ and then degrades – classic overfitting as the margin narrows.

The widening gap between training and validation accuracy beyond $C = 0.1$ clearly demonstrates overfitting: larger C values penalize margin violations more heavily, causing the model to fit noise in the training folds at the expense of generalization.

4.4.2 RBF Kernel

A 2D grid search over $C \in \{0.1, 1, 10, 100\}$ and $\gamma \in \{\text{scale}, \text{auto}, 0.001, 0.01, 0.1, 1\}$ found the best configuration at $C = 10$, $\gamma = \text{auto}$, achieving the same validation accuracy ceiling of 0.8973.

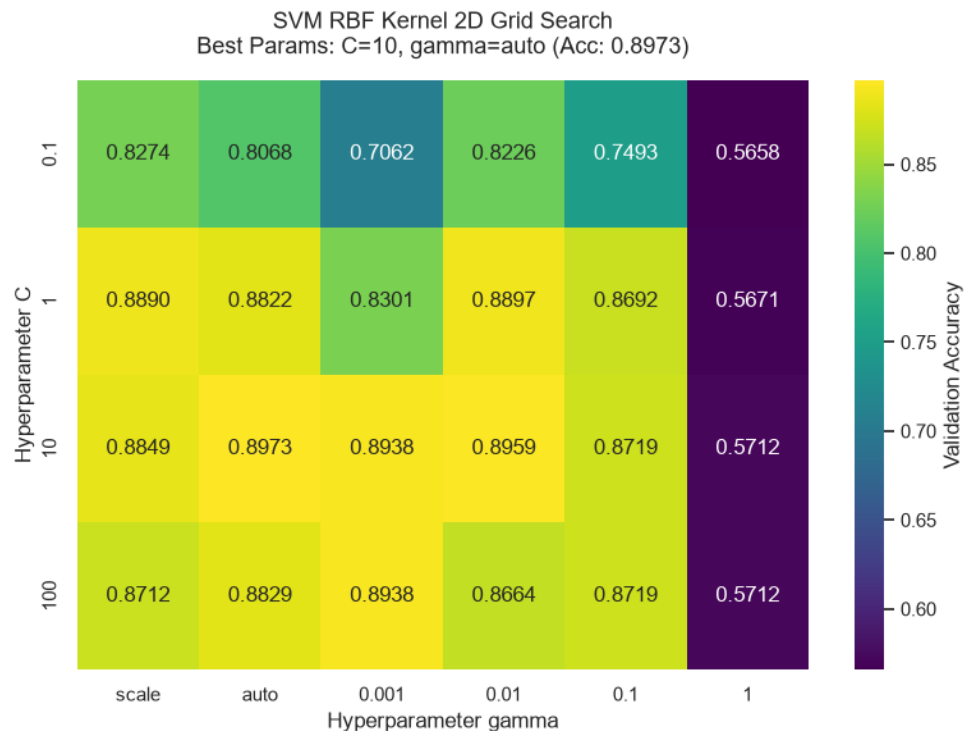


Figure 6: SVM (RBF Kernel) validation accuracy heatmap across the (C, γ) grid.

The heatmap reveals clear underfitting at $\gamma = 1$ (accuracy collapses to ~ 0.57 regardless of C , as the kernel becomes too localized) and underfitting at low C with small γ (e.g. $C = 0.1$, $\gamma = 0.001$). The best-performing region forms a plateau around $C \geq 10$ with $\gamma \in \{\text{scale}, \text{auto}, 0.01\}$, indicating a relatively wide, stable optimum rather than a single fragile best point.

4.5 Decision Trees

Cost-complexity pruning curves were generated for both the regression and classification targets.

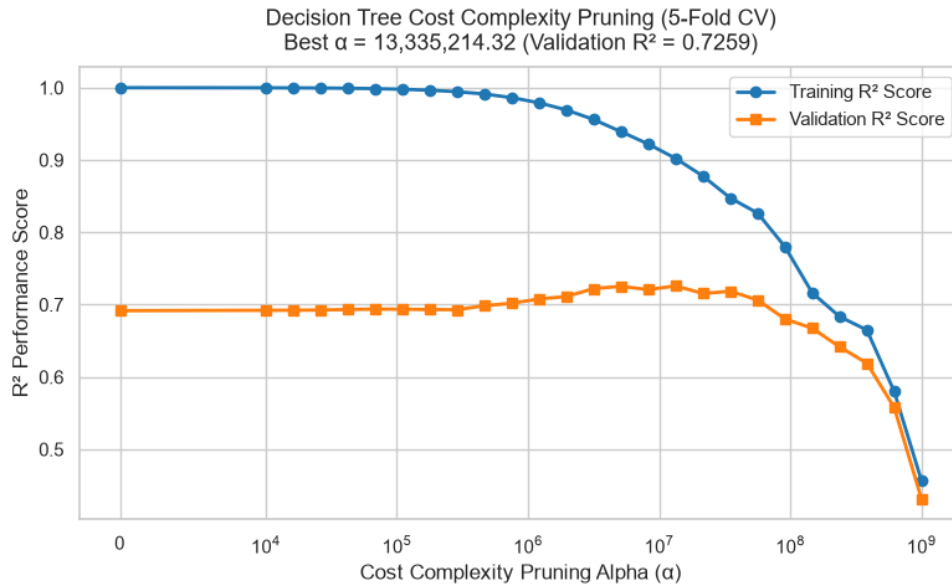


Figure 7: Decision Tree (Regressor): Training vs. Validation R^2 across pruning α . The unpruned tree ($\alpha = 0$) perfectly fits training data ($R^2=1.0$) but generalizes poorly (validation $R^2 \approx 0.69$); the optimal $\alpha \approx 1.33 \times 10^7$ raises validation R^2 to 0.7259 before both curves collapse at very high α (severe underfitting).

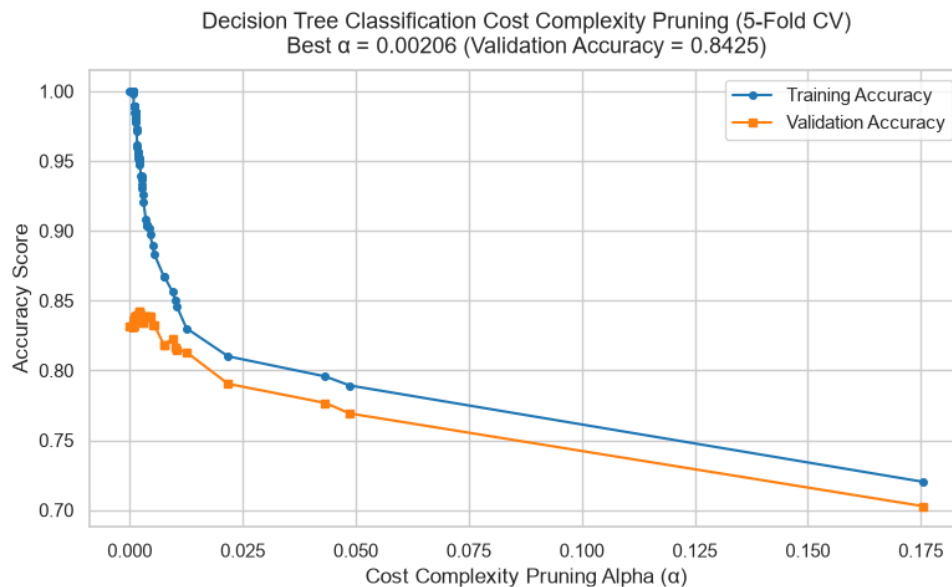


Figure 8: Decision Tree (Classifier): Training vs. Validation Accuracy across pruning α . The optimal $\alpha \approx 0.00206$ yields validation accuracy of 0.8425.

Both curves illustrate the bias–variance trade-off textbook pattern: at $\alpha = 0$ (no pruning), the tree overfits with a large train-validation gap; as α increases, the gap narrows until validation performance peaks, after which further pruning removes useful splits and both training and validation performance decline (underfitting).

4.6 Random Forest

Grid search across `n_estimators`, `max_depth`, and `max_features` (and additionally `criterion` for classification) identified the best regression configuration as `{max_depth: None, max_features: 0.3, n_estimators: 100}` ($CV R^2 = 0.8702$), and the best classification configuration as `{criterion: entropy, max_depth: 20, max_features: 0.3, n_estimators: 100}` ($CV \text{ accuracy} = 0.8932$).

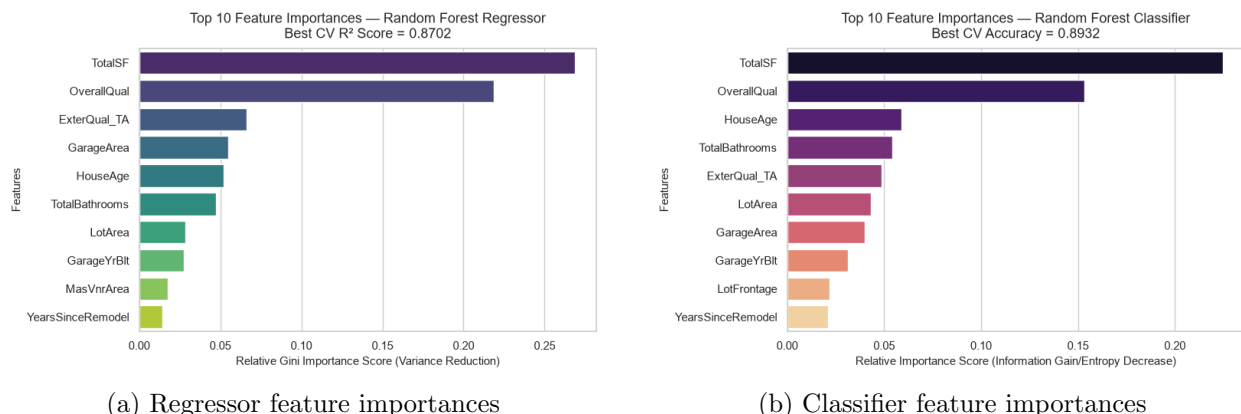


Figure 9: Random Forest top-10 feature importances for both tasks. **TotalSF** (engineered total square footage) and **OverallQual** dominate both models, confirming that size and quality are the primary drivers of Ames housing prices.

Notably, the engineered feature **TotalSF** (created in Checkpoint 2 by summing basement, first-floor, and second-floor square footage) ranks as the single most important predictor in both the regression and classification Random Forest models, validating the feature engineering decisions made earlier in the project.

4.7 Final Benchmarking Table

Table 2 consolidates results across all six models. As noted in Section 1, evaluation protocols differ slightly across models: Linear, Polynomial, and Logistic Regression report a single 80/20 train-test split; SVM, Decision Tree, and Random Forest report the best 5-fold cross-validation score from `GridSearchCV` (test-set RMSE/F1 were not separately computed for these three in the supplied notebooks).

Table 2: Final Model Benchmarking Results

Model	Regression Metric (RMSE / R^2)	Classification Metric (Accuracy / F1-Score)
Linear Regression	RMSE = \$44,025.67 ($R^2=0.7473$)	—
Logistic Regression	—	Acc. = 0.8596 ($F1_w=0.86$)
Polynomial Regression	RMSE = \$31,695.96 ($R^2=0.8690$)*	—
SVM (Linear Kernel)	—	CV Acc. = 0.8973
SVM (RBF Kernel)	—	CV Acc. = 0.8973
Decision Tree	CV R^2 = 0.7259	CV Acc. = 0.8425
Random Forest	CV R^2 = 0.8702	CV Acc. = 0.8932

*Polynomial Regression training RMSE = \$0.00 ($R^2=1.0000$), indicating severe overfitting; the test-set value above is reported for fair comparison.

Best regression model: Random Forest (CV R^2 = 0.8702), narrowly ahead of Polynomial Regression’s test R^2 of 0.8690 – though the latter’s perfect training fit makes it a far less trustworthy estimate of true generalization.

Best classification model: SVM with either kernel and Random Forest are effectively tied at the top (0.8973 and 0.8932 respectively), both outperforming Logistic Regression (0.8596) and Decision Tree (0.8425).

5 Conclusion

This capstone project implemented and benchmarked six supervised learning algorithms on the Ames Housing dataset across both a regression task (predicting `SalePrice`) and a classification task (predicting `Price_Category`). Several consistent themes emerged:

1. **Ensemble methods generalize best.** Random Forest delivered the strongest or near-strongest cross-validated performance on both tasks ($R^2=0.8702$, Accuracy=0.8932), benefiting from variance reduction via bagging and random feature subsampling – without requiring the heavy manual regularization that linear-model variants needed.
2. **Unregularized high-dimensional models overfit severely.** Polynomial Regression (degree 2) expanded the feature space from 224 to over 25,000 dimensions, achieving a perfect training fit ($R^2=1.0$) that masked a much weaker – though still respectable – test R^2 of 0.869. This is the clearest illustration in the project of why training-set performance alone is an unreliable indicator of model quality.
3. **Decision Trees require pruning to be competitive.** Unpruned single trees overfit badly; cost-complexity pruning recovered reasonable generalization ($R^2=0.7259$, Accuracy=0.8425), but single trees still underperformed the Random Forest ensemble built from many such trees, confirming the value of ensembling.
4. **SVM performance is sensitive to kernel and hyperparameter choice.** The Linear kernel’s validation accuracy peaked at a small $C = 0.1$ before degrading, while the RBF kernel exhibited sharp underfitting at extreme γ values. Both kernels converged to the same best accuracy (0.8973), suggesting the underlying class boundaries in this feature space are close to linearly separable after scaling and encoding.

5. **Feature engineering decisions paid off.** The engineered `TotalSF` feature was consistently the top predictor of house price and category across Random Forest models, validating the Checkpoint 2 decision to consolidate square-footage columns into a single derived feature.
6. **Class imbalance affects minority-class performance.** The *Premium* category, being the smallest class, consistently showed the lowest F1-scores (e.g., 0.74 in Logistic Regression), suggesting that techniques such as class weighting or resampling could be explored in future iterations to improve minority-class recall.

Overall, for deployment on unseen data, the **Random Forest** models are recommended for both tasks given their strong, well-validated cross-validated performance and resistance to overfitting relative to the high-variance Polynomial Regression and single Decision Tree alternatives. Future work could include regularized polynomial models (Ridge/Lasso), gradient boosting methods (XGBoost/LightGBM), and a unified evaluation protocol (a single held-out test set used consistently across all six models) to enable a perfectly like-for-like comparison.